

Core Data en este proyecto

Que es, como esta modelado, y como se lee y se escribe. Con el codigo real de Inventario iOS.

Ferreteria Zamora — App de gestion de inventario

Modelo: InventarioModel.xcdatamodeld | 4 entidades | Stack: NSPersistentContainer

Por que Core Data importa tanto en esta app

Core Data no es un detalle de implementacion aca: es **la fuente de verdad de toda la interfaz**.

Ninguna pantalla lee de la red. Todas leen de Core Data, y la red es un proceso aparte que lo actualiza.

Eso es lo que hace que la app funcione sin conexion, y explica por que las entidades tienen campos que no son del negocio (estadoSync, pendienteEliminar) y por que borrar no borra.

Que es Core Data, en una frase. No es una base de datos: es un **administrador de grafo de objetos** que sabe persistirse. Vos trabajas con objetos de Swift y sus relaciones; el guarda todo en un SQLite que casi nunca necesitas ver. Confundirlo con "un SQLite con azucar" lleva a pelearse con el.

PARTE I

Las piezas

Los cuatro objetos de Core Data que hay que distinguir.

1. Quien es quien

Pieza	Que es	Donde aparece
NSManagedObjectModel	El esquema: entidades, atributos y relaciones	El archivo .xcdatamodeld
NSPersistentStoreCoordinator	Conecta el modelo con el archivo en disco	No se toca nunca
NSManagedObjectContext	El area de trabajo en memoria. Todo pasa por aca	PersistenceController.viewContext
NSPersistentContainer	Arma las tres cosas anteriores por vos	PersistenceController.container

El unico que se usa a diario es el **contexto**. La imagen mental util es la de un borrador: creas y modificas objetos ahi, y nada toca el disco hasta que llamas a `save()`.

Vos escribis ■■■■■■■■■■■■■■■■■■■■	El contexto ■■■■■■■■■■■■■■■■■■■■	El disco ■■■■■■■■■■
producto.precio = 45 →	cambio pendiente	
producto.stock = 3 →	cambio pendiente	
	saveContext()	→ SQLite

2. PersistenceController

Toda la configuracion de Core Data del proyecto son 30 lineas:

CoreData/PersistenceController.swift

```
final class PersistenceController {
    static let shared = PersistenceController()

    let container: NSPersistentContainer

    private init() {
        container = NSPersistentContainer(name: "InventarioModel")
        container.loadPersistentStores { _, error in
            if let error = error {
                fatalError("No se pudo cargar Core Data: \(error)")
            }
        }
        container.viewContext.automaticallyMergesChangesFromParent = true
    }

    var viewContext: NSManagedObjectContext {
        container.viewContext
    }

    func saveContext() {
        guard viewContext.hasChanges else { return }
        do {
            try viewContext.save()
        } catch {
            assertionFailure("Error guardando Core Data: \(error)")
        }
    }
}
```

Linea por linea

- name: "InventarioModel" tiene que coincidir **exacto** con el nombre del archivo .xcdatamodeld. Si no, la app crashea al arrancar.
- loadPersistentStores abre (o crea) el SQLite. Es lo unico que puede tardar en el arranque.
- viewContext es el contexto del hilo principal. Esta app usa uno solo, lo que simplifica todo enormemente.
- guard hasChanges evita escribir al disco cuando no hay nada que escribir.

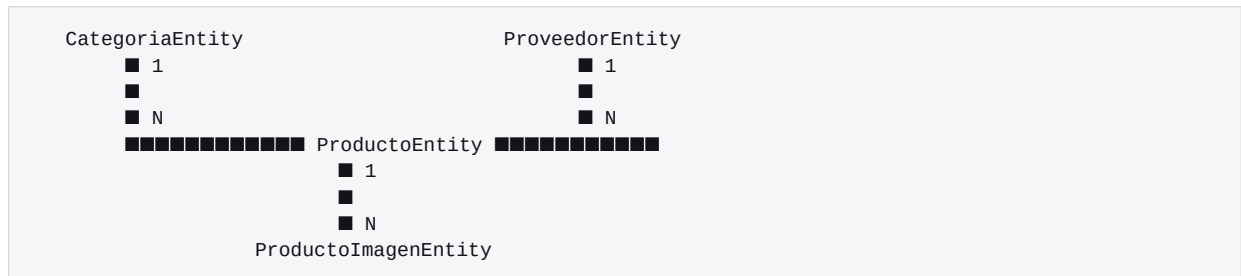
El fatalError no es descuido. Si Core Data no carga, la app no tiene fuente de datos: no hay nada util que mostrar. Arrancar igual solo lleva a un crash mas confuso tres pantallas despues. Fallar fuerte y temprano es preferible a fallar raro y tarde.

Un solo contexto, un solo hilo. Los objetos de Core Data **no son seguros entre hilos**: un NSManagedObjectContext creado en el hilo principal no se puede tocar desde otro. Esta app usa unicamente el viewContext y todos los callbacks de red vuelven al hilo principal antes de escribir. Es la razon por la que nunca aparecen crashes raros de concurrencia.

El modelo

Las cuatro entidades, sus tipos y sus relaciones.

3. El mapa de entidades



Un producto pertenece a una categoría y a un proveedor (los dos opcionales), y tiene muchas imágenes. Las relaciones son **bidireccionales**: desde una categoría se puede llegar a sus productos.

4. Los atributos de ProductoEntity

Atributo	Tipo	Opcional	Para que
nombre	String	no	Dato de negocio
precio	Double	no	Dato de negocio
stock	Integer 32	no	Dato de negocio
fechaRegistro	Date	si	Dato de negocio
localId	String	no	Control de sync
apiId	Integer 64	si	Control de sync
estadoSync	Integer 16	no	Control de sync
pendienteEliminar	Boolean	no	Control de sync

Los cuatro últimos no existen en el backend: son **de la app**, y están para sostener el modo offline. Se explican en la Parte IV.

El detalle que confunde a todos: usesScalarValueType

En el XML del modelo hay una diferencia sutil entre dos atributos numéricos:

InventarioModel.xcdatamodel/contents

```

<attribute name="apiId" optional="YES" attributeType="Integer 64" usesScalarValueType="NO"/>
<attribute name="estadoSync" attributeType="Integer 16" usesScalarValueType="YES"/>

```

Esa bandera decide el tipo que Xcode genera en Swift:

usesScalarValueType	Tipo en Swift	Puede ser nil
YES (escalar)	Int16	No
NO (objeto)	NSNumber?	Si

Por eso `apiId` puede estar vacío — un producto que todavía no se subió no tiene id de servidor — y por eso en el código aparece esta forma incómoda:

```
producto.apiId = NSNumber(value: dto.id) // al escribir: hay que envolver
let id = apiId.int64Value               // al leer: hay que desenvolver
```

La regla práctica. Si el número **puede faltar**, va como objeto (`usesScalarValueType=NO`) y en Swift es `NSNumber?`. Si siempre tiene valor, va escalar y queda como `Int16` o `Double`, mucho más cómodo. Un escalar no puede ser nil aunque marques "optional" en el editor: Core Data le pone 0.

5. Las relaciones y sus reglas de borrado

Cada relación define qué pasa con el otro lado cuando se borra un objeto. El proyecto usa dos reglas distintas, y la elección no es casual:

Relación	Regla	Que significa
Producto → imágenes	Cascade	Al borrar el producto se borran sus imágenes. Una imagen sin producto no tiene sentido: sería basura
Producto → categoría	Nullify	Al borrar la categoría, los productos quedan sin categoría pero siguen existiendo
Producto → proveedor	Nullify	Igual: se pierde el vínculo, no el producto

```
<relationship name="imagenes" toMany="YES" deletionRule="Cascade"
  destinationEntity="ProductoImagenEntity" inverseName="producto"/>

<relationship name="categoria" maxCount="1" deletionRule="Nullify"
  destinationEntity="CategoriaEntity" inverseName="productos"/>
```

Si te equivocas de regla, pierdes datos. Poner Cascade en la categoría significaría que borrar una categoría borra todos sus productos. Es un error silencioso y catastrófico: nadie lo nota hasta que falta media tabla. La UI de la app además lo advierte — al borrar una categoría avisa que "los productos que la usen quedan sin categoría".

inverseName: siempre las dos puntas

Toda relación declara su inversa. Core Data usa eso para mantener la coherencia automáticamente: al hacer `producto.categoria = X`, el producto aparece solo dentro de `X.productos`. No hay que actualizar los dos lados a mano.

6. Las clases las genera Xcode

```
<entity name="ProductoEntity" representedClassName="ProductoEntity"
  syncable="YES" codeGenerationType="class">
```

codeGenerationType="class" significa que **no hay ningun archivo** ProductoEntity.swift **escrito a mano** en el proyecto. Xcode lo genera al compilar, a partir del modelo.

Consecuencias practicas:

- Si buscas ProductoEntity.swift en el repositorio, **no lo vas a encontrar**. Es normal.
- Agregar un atributo en el editor visual alcanza para poder usarlo en Swift.
- No se le pueden agregar metodos propios a la entidad por ese camino; para eso habria que cambiar a generacion manual o usar una extension.
- Es la razon de una rareza del codigo que se explica en la Parte V.

PARTE III

Leer

Consultas: NSFetchRequest, predicados y orden.

7. Anatomia de una consulta

Services/ProductoService.swift

```
func productosLocales() -> [ProductoEntity] {
    let request = NSFetchRequest<ProductoEntity>(entityName: "ProductoEntity")
    request.predicate = NSPredicate(format: "pendienteEliminar == NO")
    request.sortDescriptors = [
        NSSortDescriptor(
            key: "nombre",
            ascending: true,
            selector: #selector(NSString.localizedCaseInsensitiveCompare(_:_))
        )
    ]
    return (try? contexto.fetch(request)) ?? []
}
```

Si lo piensas en SQL, la traduccion es directa:

SELECT * FROM ProductoEntity	← NSFetchRequest(entityName:)
WHERE pendienteEliminar = NO	← request.predicate
ORDER BY nombre COLLATE NOCASE ASC	← request.sortDescriptors

El predicado

NSPredicate se escribe con un formato de texto propio. Cuando hay valores variables, van con marcadores y **nunca** concatenando texto:

```
NSPredicate(format: "pendienteEliminar == NO")           // literal
NSPredicate(format: "apiId == %@", NSNumber(value: apiId)) // con valor
NSPredicate(format: "estadoSync == 0 AND pendienteEliminar == NO") // compuesto
```

El %@ toma un objeto, por eso el NSNumber. Para numeros escalares se usa %d. Es el mismo motivo por el que en SQL se usan parametros: evita errores de formato y de escapado.

El orden y el detalle del selector

El selector: no es adorno:

```
selector: #selector(NSString.localizedCaseInsensitiveCompare(_:))
```

Sin ese selector, el orden queda mal. La comparacion por defecto es binaria: ordena por el valor numerico de cada caracter. Eso pone todas las mayusculas antes que las minusculas ("Zinc" antes que "acero") y manda las palabras con tilde al final. `localizedCaseInsensitiveCompare` ordena como espera una persona.

Por que try? y no do/catch

(try? ...) ?? [] convierte cualquier error de consulta en lista vacia. Es deliberado: si la lectura local falla, mostrar una lista vacia es mejor que crashear la app. El usuario puede sincronizar y recuperarse; un crash no deja salida.

8. Buscar uno solo

Sync/SyncManager.swift

```
private func buscarPorApiId<T: NSManagedObject>(  
    _ tipo: T.Type, nombreEntidad: String, apiId: Int64  
) -> T? {  
    let request = NSFetchRequest<T>(entityName: nombreEntidad)  
    request.predicate = NSPredicate(format: "apiId == %@", NSNumber(value: apiId))  
    request.fetchLimit = 1  
    return try? contexto.fetch(request).first  
}
```

`fetchLimit = 1` es una optimizacion real: le dice a Core Data que pare de buscar apenas encuentre uno, en vez de traer todos los que coincidan y quedarse con el primero.

Esta funcion es el puente entre los dos mundos de ids: recibe el `apiId` que mando el servidor y devuelve la fila local correspondiente.

9. Navegar relaciones

Las relaciones se leen como propiedades comunes:

```
producto.categoria?.nombre          // subir por la relacion  
producto.proveedor?.logoUrl
```

Pero las relaciones **a muchos** tienen una trampa: Core Data las entrega como `NSSet`, que es un **conjunto sin orden**.

Features/Productos/ProductoTableViewCell.swift

```
let imagenes = (producto.imagenes as? Set<ProductoImagenEntity>) ?? []  
let principal = imagenes.sorted { $0.orden < $1.orden }.first
```

Un Set no tiene primer elemento. Si tomaras `imagenes.first` directamente, la "foto principal" podría cambiar entre dos aperturas de la misma pantalla, sin que nadie haya tocado nada. Por eso existe el atributo `orden` y por eso siempre se ordena antes de usar. El mismo criterio se aplica en la galería del detalle y en la miniatura del listado, para que las dos coincidan.

PARTE IV

Escribir

Crear, actualizar, borrar, y los cuatro campos de control.

10. Los cuatro campos de control

Campo	Valores	Significado
<code>localId</code>	UUID	Identificador propio de la app. Existe desde el instante en que se crea la fila, incluso sin conexión
<code>apiId</code>	Int64 o nil	Id que asigno el servidor. nil = nunca se subió
<code>estadoSync</code>	0 o 1	0 = tiene cambios sin subir (chip PENDIENTE). 1 = igual que en el servidor
<code>pendienteEliminar</code>	true / false	El usuario lo borra; el DELETE al servidor todavía no se confirmó

Las cuatro entidades tienen estos cuatro campos. Es el precio de que la app funcione sin conexión: hay que recordar en el propio dato en que estado está respecto del servidor.

11. Crear

`Services/ProductoService.swift`

```
let producto = ProductoEntity(context: contexto)
producto.localId = UUID().uuidString
// apiId queda nil: todavía no existe en el servidor. El SyncManager lo
// completa cuando el POST devuelve el id asignado.
producto.apiId = nil
producto.estadoSync = 0
producto.pendienteEliminar = false
producto.fechaRegistro = Date()

aplicar(nombre: nombre, precio: precio, stock: stock, ...)
PersistenceController.shared.saveContext()
```

`ProductoEntity(context:)` **inserta el objeto en el contexto** en el acto. No hace falta un "insert" aparte: crear el objeto ya lo agrega.

12. Actualizar

No hay ningún UPDATE. Se modifican las propiedades del objeto y se guarda:

```
private func aplicar(..., a producto: ProductoEntity) {
    producto.nombre = nombre
    producto.precio = precio
    producto.stock = stock
    producto.categoria = categoria
    producto.proveedor = proveedor
    // Marca la fila como pendiente de subir. Tambien protege el cambio: el
    // upsert de la bajada saltea todo lo que tenga estadoSync == 0.
    producto.estadoSync = 0
}
```

Core Data lleva la cuenta de que cambio (hasChanges) y en el save () escribe solo lo necesario.

Ese estadoSync = 0 hace dos trabajos. El obvio: marcar la fila para que la proxima sincronizacion la suba, y dibujar el chip naranja. El menos obvio: **protege el cambio**. Cuando despues se bajan los datos del servidor, el upsert saltea todo lo que tenga estadoSync == 0, asi que la version del servidor no puede pisar lo que el usuario acaba de escribir sin conexion.

13. Borrar: por que casi nunca se borra

```
func marcarParaEliminar(_ producto: ProductoEntity) {
    if producto.apiId == nil {
        contexto.delete(producto) // nunca llego al servidor: se va de verdad
    } else {
        producto.pendienteEliminar = true // borrado logico
        producto.estadoSync = 0
    }
    PersistenceController.shared.saveContext()
}
```

El borrado es **logico**. La fila se marca y sigue en la base hasta que el servidor confirme el DELETE. Para el usuario desaparece igual, porque todas las consultas filtran con pendienteEliminar == NO.

Que pasaria sin el borrado logico. Si la fila se borrara del telefono al instante y la app estuviera sin conexion, no quedaria ningun rastro de que hay que avisarle al servidor. En la proxima sincronizacion el producto volveria a bajar, intacto, como si nunca lo hubieras borrado.

14. El upsert: la escritura mas delicada

Al bajar del servidor hay que actualizar lo que ya existe e insertar lo nuevo. La busqueda es por apiId, el unico id que las dos puntas comparten:


```
private func entidadParaUpsert<T: NSManagedObject>(
    _ tipo: T.Type, nombreEntidad: String, apiId: Int64
) -> T? {
    if let existente = buscarPorApiId(tipo, nombreEntidad: nombreEntidad, apiId: apiId) {
        let estadoSync = existente.value(forKey: "estadoSync") as? Int16 ?? 0
        let pendienteEliminar = existente.value(forKey: "pendienteEliminar") as? Bool ?? false
        guard estadoSync == 1, !pendienteEliminar else { return nil }
        return existente
    }

    let nueva = T(context: contexto)
    nueva.setValue(UUID().uuidString, forKey: "localId")
    nueva.setValue(NSNumber(value: apiId), forKey: "apiId")
    nueva.setValue(Int16(1), forKey: "estadoSync")
    nueva.setValue(false, forKey: "pendienteEliminar")
    return nueva
}
```

El guard `estadoSync == 1, !pendienteEliminar` es la clave. Si la fila local tiene cambios sin subir o esta marcada para borrar, la funcion devuelve `nil` y el que llama la saltea. **Nunca se pisa trabajo offline.**

La entidad nueva nace con `estadoSync = 1` porque viene del servidor: por definicion ya esta sincronizada.

Reconciliar lo borrado desde afuera

```
/// Borra lo que alguien elimino desde el portal web. Sin esto, un producto
/// borrado en el servidor sobreviviria para siempre en el telefono.
private func eliminarLocalesQueElServidorYaNoTiene(_ nombreEntidad: String,
                                                    apiIds: Set<Int64>) {
    let request = NSFetchRequest<NSManagedObject>(entityName: nombreEntidad)
    request.predicate = NSPredicate(format: "estadoSync == 1 AND apiId != nil")
    ...
}
```

El predicado vuelve a proteger lo pendiente: solo se miran filas ya sincronizadas y con `apiId`. Lo que el usuario acaba de crear sin conexion no aparece en el servidor *todavia*, y borrarlo por eso seria un desastre.

Un caso especial: las imagenes

```
/// Las imagenes todavia no se editan localmente (Fase 5), asi que la version
/// del servidor es la unica verdad: se borran las locales y se reinsertan.
private func reemplazarImagenes(de producto: ProductoEntity, con dtos: [ImagenDTO]) {
    if let existentes = producto.imagenes as? Set<ProductoImagenEntity> {
        existentes.forEach { contexto.delete($0) }
    }

    for dto in dtos {
        let imagen = ProductoImagenEntity(context: contexto)
        imagen.localId = UUID().uuidString
        imagen.apiId = NSNumber(value: dto.id)
        ...
        imagen.producto = producto // basta con setear un lado
    }
}
```

Aca si se reemplaza todo, y es correcto: las imagenes no se editan sin conexion, asi que no hay trabajo offline que proteger. La ultima linea muestra la relacion bidireccional trabajando: al asignar `imagen.producto`, esa imagen aparece sola dentro de `producto.imagenes`.

PARTE V

Rarezas del código

Dos cosas que sorprenden al leer, y por que están.

15. Por que aparece setValue(forKey:)

En algunos lugares el código no usa propiedades sino **KVC** (acceso por nombre de campo, en texto):

```
nueva.setValue(UUID().uuidString, forKey: "localId")
let estadoSync = existente.value(forKey: "estadoSync") as? Int16 ?? 0
```

Parece un retroceso — se pierde el chequeo del compilador — pero tiene una razón concreta, explicada en el propio código:

Services/CatalogoService.swift

```
/// Lo que categorías y proveedores hacen igual. Se accede por KVC porque las dos
/// entidades comparten los atributos de control pero no un tipo común: las
/// clases las genera `momc` desde el `.xcdatamodeld` y no hay donde meter un
/// protocolo a mano.
enum Catalogo {
```

Las cuatro entidades tienen localId, apiId, estadoSync y pendienteEliminar, pero como las clases las genera Xcode, no hay un archivo donde hacerlas conformar a un protocolo común. Sin tipo común, el código genérico no puede escribir entidad.estadoSync.

El intercambio, dicho claro. Se gana no repetir la misma lógica cuatro veces; se pierde el chequeo del compilador. Si alguien renombra estadoSync en el editor visual, el código **compila igual** y falla en ejecución. La alternativa — generación manual de las clases y un protocolo — era más código del que ahorra.

16. Los servicios son struct, no class

```
struct ProductoService {
    private var contexto: NSManagedObjectContext { PersistenceController.shared.viewContext }
```

Se instancian donde se usan (ProductoService().todos()) sin guardarlos en ninguna propiedad. Pueden ser struct porque **no tienen estado**: el estado está en Core Data, no en el servicio. Cada instancia es un envoltorio alrededor del mismo contexto compartido.

PARTE VI

Práctica

Como mirar los datos y que errores evitar.

17. Inspeccionar la base a mano

Core Data guarda en un SQLite comun. Durante el desarrollo se puede abrir y consultar directamente, que es la forma mas confiable de verificar que algo se guardo:

Terminal

```
# Ubicar el archivo del simulador
find ~/Library/Developer/CoreSimulator/Devices -name "InventarioModel.sqlite" 2>/dev/null

# Consultarlo
sqlite3 <ruta> "SELECT ZNOMBRE, ZPRECIO, ZSTOCK, ZESTADOSYNC FROM ZPRODUCTOENTITY;"
```

Los nombres llevan Z adelante. Core Data prefija tablas y columnas con Z: ProductoEntity se convierte en ZPRODUCTOENTITY, y nombre en ZNOMBRE. Es un detalle interno — no hay que asumir que ese esquema es estable ni escribir en el a mano.

18. Errores clasicos que este proyecto evita

Error	Que pasa	Como se evita aca
Olvidarse el save()	El cambio se ve en pantalla y desaparece al reiniciar	Todos los metodos de escritura llaman a saveContext() al final
Tocar objetos desde otro hilo	Crashes raros e intermitentes	Un solo contexto y todos los callbacks vuelven al hilo principal
Cascade donde va Nullify	Borrar una categoria borra sus productos	Cascade solo en imagenes, que sin producto no tienen sentido
Usar el primer elemento de un Set	El orden cambia sin razon aparente	Se ordena por el atributo orden antes de usar
Cambiar el modelo sin migracion	La app crashea al abrir con datos viejos	Ver la nota de abajo

Sobre cambiar el modelo

Agregar o renombrar atributos en un modelo que ya tiene datos guardados **rompe la app** si no se hace una migracion. Durante el desarrollo la salida practica es borrar la app del simulador y reinstalarla, que descarta el store viejo.

En una app publicada eso no es opcion y hay que usar migracion (ligera o con mapeo). Este proyecto no la necesito porque el modelo quedo definido antes de tener datos que valiera la pena conservar.

19. Resumen

Para...	Se usa
Leer una lista	NSFetchRequest + predicate + sortDescriptors
Leer uno solo	El mismo, con fetchLimit = 1
Crear	Entidad(context:) y setear propiedades

Para...	Se usa
Actualizar	Modificar propiedades del objeto
Borrar de verdad	contexto.delete(objeto)
Borrar sincronizable	pendienteEliminar = true (borrado logico)
Confirmar cambios	PersistenceController.shared.saveContext()
Relacion a uno	producto.categoria
Relacion a muchos	producto.imagenes as? Set<...>, y ordenar

La idea que ordena todo. Core Data aca no es "donde se guarda la copia": es **la fuente de verdad de la interfaz**. La red actualiza Core Data; la pantalla lee Core Data. Esa direccion unica es lo que hace que la app funcione sin conexion, y todo lo demas — los cuatro campos de control, el borrado logico, los guardas del upsert — existe para sostenerla.